

UNIVERSIDAD DEL VALLE DE GUATEMALA

1CC30642020261 – Computación Paralela y Distribuida

Sección 10

Catedrática: Kimberly Barrera

Proyecto 1

Screensaver de chuchos paralelizado con OpenMP

Isabella Obando – 23074

Anthony Lou – 23410

Nelson Escalante – 22046

Guatemala, 23 de septiembre de 2026

Índice

1. Introducción	2
2. Antecedentes	2
2.1. OpenMP y memoria compartida	2
2.2. Metodología PCAM	2
2.3. Speedup, eficiencia y ley de Amdahl	2
2.4. SDL2	3
3. Desarrollo	3
3.1. Descripción del screensaver	3
3.2. Modelo físico	4
3.3. Versión secuencial	4
3.4. Diseño paralelo con PCAM	4
3.5. Secciones paralelas y sincronización	5
3.6. Zona paralela visual	5
3.7. Metodología de medición	6
4. Resultados	6
4.1. Discusión	7
5. Conclusiones	8
6. Recomendaciones	8
A. Anexo 1 – Diagrama de flujo del programa	10
B. Anexo 2 – Catálogo de funciones	11
C. Anexo 3 – Bitácora de pruebas	14
D. Material suplementario	16

1. Introducción

Este informe documenta el diseño, la implementación y la evaluación de un *screensaver* escrito en C++ con la biblioteca gráfica SDL2 y paralelizado con OpenMP. El programa dibuja N “chuchos” (sprites de perros de 64×64 píxeles) de colores pseudoaleatorios que se desplazan por un canvas de 640×480 , rebotan contra los bordes y chocan entre sí con una respuesta basada en choques elásticos. El valor de N se recibe por línea de comandos y en pantalla se muestran los FPS junto con el tiempo que tarda la simulación.

El trabajo siguió el enfoque iterativo que propone OpenMP: primero se construyó una versión secuencial funcional y luego se paralelizaron, una por una, las regiones de mayor costo, verificando en cada paso que el resultado no cambiara. Como ayuda visual, la pantalla se divide en una zona secuencial y una zona paralela de otro color, cuyo tamaño se puede ajustar en vivo.

Además del screensaver se construyó un programa de *benchmark* sin ventana que mide la simulación para distintos valores de N y de hilos, calcula el *speedup* y la eficiencia, y verifica bit a bit que la versión paralela produzca exactamente el mismo resultado que la secuencial.

2. Antecedentes

2.1. OpenMP y memoria compartida

OpenMP es una interfaz de programación para paralelismo en memoria compartida basada en directivas de compilador, rutinas de biblioteca y variables de entorno [4]. Su modelo de ejecución es *fork-join*: un hilo maestro crea un equipo de hilos al entrar a una región paralela y, al salir, todos se sincronizan en una barrera implícita antes de que continúe solo el maestro [2]. La directiva `parallel for` reparte las iteraciones de un ciclo entre los hilos según una política de *scheduling* (`static`, `dynamic`, `guided`), y la cláusula `if` permite decidir en tiempo de ejecución si una región se ejecuta en paralelo o con un solo hilo¹.

2.2. Metodología PCAM

Foster propone diseñar algoritmos paralelos en cuatro etapas [3]: **Partición** (descomponer el cálculo y los datos en tareas pequeñas), **Comunicación** (identificar qué datos necesita cada tarea de las demás), **Aglomeración** (agrupar tareas para reducir el costo de comunicación y sincronización) y **Mapeo** (asignar los grupos a procesadores). Este proyecto aplica PCAM a la simulación de los chuchos en la sección 3.4.

2.3. Speedup, eficiencia y ley de Amdahl

Si T_s es el tiempo de la mejor versión secuencial y T_p el tiempo de la versión paralela con p hilos, el *speedup* y la eficiencia son

$$S(p) = \frac{T_s}{T_p}, \quad E(p) = \frac{S(p)}{p}. \quad (1)$$

Amdahl mostró que si una fracción f del programa es inherentemente secuencial, el speedup está acotado por [1, 5]

$$S(p) \leq \frac{1}{f + \frac{1-f}{p}}. \quad (2)$$

¹Con `if(false)` la región se ejecuta con un equipo de un solo hilo, lo que permite usar el mismo código para la versión secuencial y la paralela.

En este informe se reportan dos speedups: contra la mejor versión secuencial (S_{sec}) y contra la misma versión paralela con un hilo (S_{1h}), porque, como se verá, la versión paralela realiza el doble de revisiones de colisión que la secuencial.

2.4. SDL2

SDL (*Simple DirectMedia Layer*) es una biblioteca multiplataforma en C que da acceso a ventanas, eventos de teclado y ratón, y renderizado 2D acelerado [6]. En este proyecto se usa para crear la ventana, generar la textura del chuchos a partir de un bitmap, teñirla con `SDL_SetTextureColorMod` y dibujar el HUD con una fuente de píxeles propia².

3. Desarrollo

3.1. Descripción del screensaver

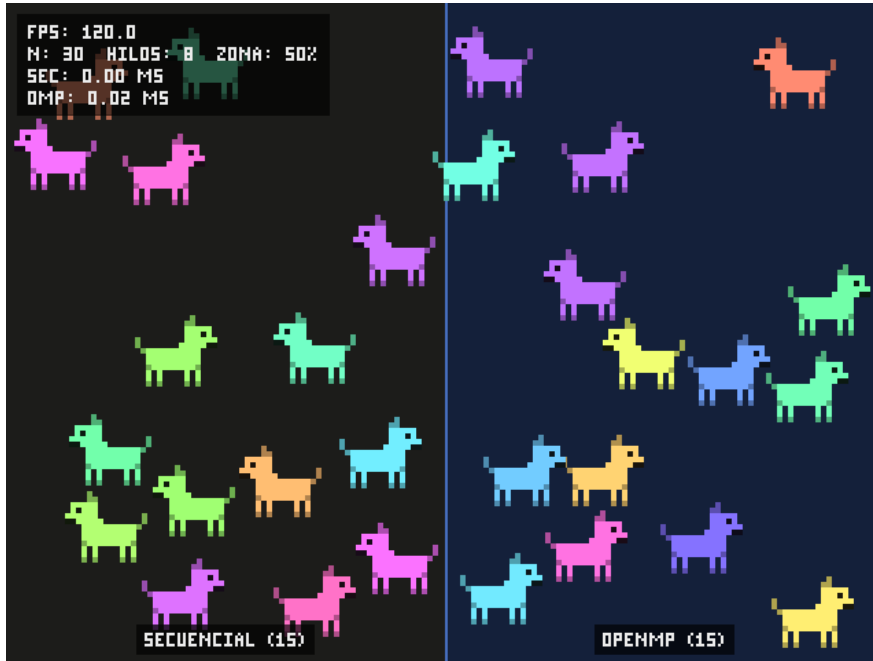


Figura 1: Screensaver con la zona paralela al 50 %: a la derecha (fondo azul) los chuchos se procesan con OpenMP; el HUD muestra FPS, N , hilos y los milisegundos por frame de cada zona.

La Tabla 1 resume cómo se cumple cada requisito del enunciado.

²Se evitó depender de `SDL2_ttf` para que el proyecto solo requiera `libsdl2-dev`.

Tabla 1: Requisitos del screensaver y su implementación

Requisito	Implementación
Parámetro N	<code>./screensaver [N] [semilla] [--zona P] [--hilos H];</code> todos se validan.
Colores pseudoaleatorios	Tono aleatorio (HSV) por chucho con <code>std::mt19937</code> ; cambia al chocar.
Canvas $\geq 640 \times 480$	Ventana de 640×480 .
Movimiento	Velocidad aleatoria en magnitud (60–240 px/s) y dirección.
Física / trigonometría	Integración explícita, rebote en bordes, colisiones elásticas entre chuchos.
Despliegue de FPS	HUD con FPS, ms por zona, N , hilos y tamaño de la zona.

3.2. Modelo físico

Cada chucho guarda su posición (x, y) , velocidad (v_x, v_y) , tono y tinte. En cada frame la posición se integra de forma explícita con un Δt acotado a 0.05 s:

$$x \leftarrow x + v_x \Delta t, \quad y \leftarrow y + v_y \Delta t. \quad (3)$$

Si el sprite sale del rango válido ($0 \leq x \leq 576$, $0 \leq y \leq 416$), la posición se refleja respecto al borde y la componente de la velocidad se fuerza a apuntar hacia adentro, lo que evita que el chucho quede “pegado” a la pared.

Dos chuchos a y b **chocan** si sus cajas envolventes (56×52 px) se traslapan y además se están acercando:

$$|x_a - x_b| < 56 \wedge |y_a - y_b| < 52 \wedge (\mathbf{p}_b - \mathbf{p}_a) \cdot (\mathbf{v}_b - \mathbf{v}_a) < 0. \quad (4)$$

La condición del producto punto evita que un par que sigue traslapado vuelva a chocar en el frame siguiente. Si el chucho i chocó con k compañeros en el frame, su nueva velocidad toma la *dirección* del promedio de las velocidades de los compañeros y la *rapidez* promedio de ellos:

$$\mathbf{v}_i \leftarrow \frac{\bar{\mathbf{v}}}{\|\bar{\mathbf{v}}\|} \frac{1}{k} \sum_j \|\mathbf{v}_j\|, \quad \bar{\mathbf{v}} = \frac{1}{k} \sum_j \mathbf{v}_j. \quad (5)$$

Con un solo compañero esto equivale a intercambiar velocidades (choque elástico entre masas iguales). Se descartó promediar los vectores directamente porque en pruebas le quitaba energía al sistema: con $N \geq 100$ la rapidez promedio caía de ~ 150 a 0 px/s en un minuto.

3.3. Versión secuencial

La versión secuencial detecta colisiones recorriendo el triángulo superior de pares ($i < j$): $N(N-1)/2$ revisiones. Cada choque se anota en un buffer (suma de velocidades, suma de rapideces y cantidad de choques de cada chucho) y en una segunda pasada se aplica. Separar *detectar* de *aplicar* hace que el resultado no dependa del orden en que se revisan los pares.

3.4. Diseño paralelo con PCAM

Partición. Se usó descomposición de dominio: la tarea mínima es “procesar al chucho i ” en cada una de las tres etapas del frame (mover, detectar choques, aplicar choques). Mover y aplicar son independientes por chucho. Detectar es $O(N)$ por chucho y $O(N^2)$ en total, y concentra entre el 93% y el 99% del tiempo (Tabla 3).

Comunicación. Para detectar, la tarea i necesita leer la posición y velocidad de todos los demás chuchos (comunicación global de solo lectura). En la versión de triángulo, un choque entre i y j escribe en las entradas i y j del buffer; en paralelo, dos hilos podrían escribir la misma entrada j al mismo tiempo (condición de carrera).

Aglomeración. Para eliminar la condición de carrera sin *locks* ni operaciones atómicas, cada tarea i recorre la fila completa ($j = 0, \dots, N - 1, j \neq i$), acumula en variables locales y escribe únicamente su propia entrada del buffer. El costo es duplicar las revisiones ($N(N - 1)$ en lugar de $N(N - 1)/2$). Como cada chuchito suma a sus compañeros en el mismo orden que en la versión de triángulo, el resultado es idéntico bit a bit.

Mapeo. Las iteraciones se reparten con `schedule(static)`: al recorrer la fila completa todas cuestan lo mismo, así que bloques contiguos e iguales balancean la carga sin costo de coordinación. El número de hilos se toma de `--hilos` o del valor por defecto de OpenMP (8 en el equipo de pruebas).

3.5. Secciones paralelas y sincronización

Las tres etapas usan la misma directiva, con la cláusula `if` para decidir si corren en paralelo:

```
#pragma omp parallel for schedule(static) if (paralelo)
for (int k = 0; k < m; ++k) {
    const int i = indices[k];    // cada hilo escribe solo en sus propios i
    ...
}
```

Los mecanismos de sincronización y protección de memoria compartida son:

- **Barreras implícitas** al final de cada `parallel for`: garantizan que todos los chuchos se movieron antes de detectar, y que todos terminaron de detectar antes de aplicar (aplicar modifica las velocidades que detectar lee).
- **Propiedad exclusiva de datos**: en cada etapa el hilo que procesa al chuchito i es el único que escribe su estado y su entrada del buffer; las lecturas a los demás chuchos ocurren en una etapa donde nadie los modifica.
- **Acumuladores locales** (privados de cada hilo) durante la detección, escritos al buffer compartido una sola vez al final.

Los buffers se reservan una vez y se reutilizan entre frames (`std::vector`); SDL se inicializa y se destruye en orden inverso, y cada error de inicialización libera lo ya creado antes de salir.

3.6. Zona paralela visual

Para visualizar la paralelización, al inicio de cada frame los chuchos se clasifican según la posición de su centro: los que están en la franja derecha (la zona paralela, de fondo azul) se procesan con OpenMP y el resto con un solo hilo. El porcentaje de la zona se elige con `--zona P` (0 = todo secuencial, 100 = todo paralelo) o en vivo con las flechas, y el HUD muestra cuánto tarda cada zona por frame (Figura 1).

3.7. Metodología de medición

Las mediciones se hicieron con el programa `build/benchmark`, que corre la simulación *sin ventana* para que el dibujo no contamine los tiempos:

- Equipo: Intel Core i5-1035G1 (4 núcleos físicos, 8 hilos lógicos), Ubuntu 24.04 en WSL2, g++ 13.3 con `-O2 -fopenmp`.
- $N \in \{500, 1000, 2000, 4000\}$ y hilos $\in \{1, 2, 4, 8\}$.
- Por prueba: 10 repeticiones; cada una recrea la escena con la semilla 7, corre 5 frames de calentamiento y mide 30 frames con Δt fijo de 1/60 s. Se reporta el promedio de ms por frame.
- Al terminar, el estado final de cada corrida paralela se compara bit a bit con el de la secuencial.

El FPS del screensaver real se midió aparte, con 8 segundos por corrida y `OMP_WAIT_POLICY=passive` (ver sección 4.1).

4. Resultados

Tabla 2: Tiempo de simulación por frame, speedup contra la mejor versión secuencial (S sec), contra OpenMP con un hilo (S 1h) y eficiencia E .

N	Versión	Hilos	ms/frame	Desv.	S (sec)	S (1h)	E
500	secuencial	–	0.503	0.019	1.00	–	1.00
500	openmp	1	1.031	0.059	0.49	1.00	0.49
500	openmp	2	0.542	0.059	0.93	1.90	0.46
500	openmp	4	0.335	0.053	1.50	3.08	0.37
500	openmp	8	0.253	0.082	1.99	4.08	0.25
1000	secuencial	–	1.926	0.052	1.00	–	1.00
1000	openmp	1	3.780	0.102	0.51	1.00	0.51
1000	openmp	2	2.064	0.184	0.93	1.83	0.47
1000	openmp	4	1.501	0.102	1.28	2.52	0.32
1000	openmp	8	1.064	0.239	1.81	3.55	0.23
2000	secuencial	–	8.216	0.429	1.00	–	1.00
2000	openmp	1	16.472	0.913	0.50	1.00	0.50
2000	openmp	2	9.025	0.316	0.91	1.83	0.46
2000	openmp	4	6.167	0.149	1.33	2.67	0.33
2000	openmp	8	4.724	1.075	1.74	3.49	0.22
4000	secuencial	–	36.187	1.995	1.00	–	1.00
4000	openmp	1	73.634	9.150	0.49	1.00	0.49
4000	openmp	2	46.585	8.269	0.78	1.58	0.39
4000	openmp	4	33.432	2.381	1.08	2.20	0.27
4000	openmp	8	27.433	3.326	1.32	2.68	0.16

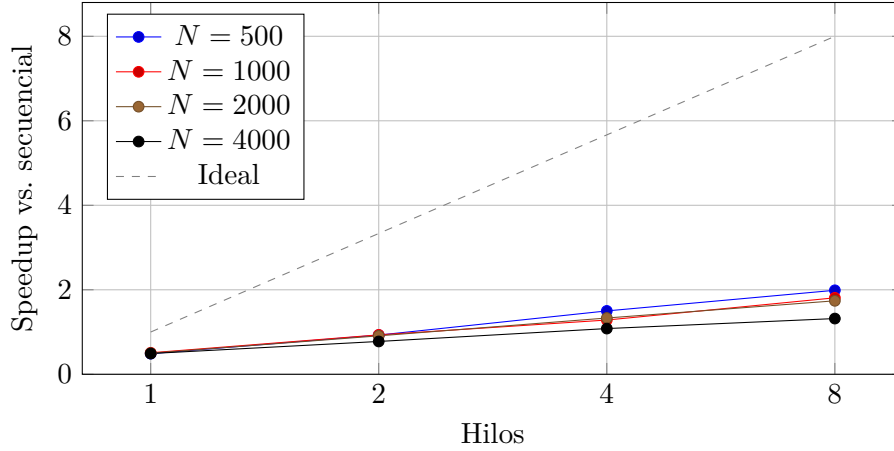


Figura 2: Speedup de la versión OpenMP respecto a la mejor versión secuencial.

Tabla 3: Tiempo por etapa (ms por frame): secuencial y OpenMP con 8 hilos.

N	Versión	Mover (ms)	Detectar (ms)	Aplicar (ms)	% detectar
500	Secuencial	0.003	0.468	0.032	93.0 %
500	OpenMP 8h	0.008	0.204	0.040	80.8 %
1000	Secuencial	0.006	1.856	0.064	96.3 %
1000	OpenMP 8h	0.046	0.910	0.108	85.5 %
2000	Secuencial	0.012	8.075	0.129	98.3 %
2000	OpenMP 8h	0.164	4.246	0.314	89.9 %
4000	Secuencial	0.029	35.867	0.291	99.1 %
4000	OpenMP 8h	0.766	25.537	1.130	93.1 %

Tabla 4: FPS del screensaver real (incluye dibujar) con 8 hilos. En el screensaver, la zona 0 % usa el mismo recorrido completo que OpenMP pero con un solo hilo, por eso sus ms de simulación son mayores que los de la versión secuencial del benchmark.

N	Secuencial (zona 0 %)		OpenMP (zona 100 %)		Mejora FPS
	FPS	ms sim.	FPS	ms sim.	
250	246.0	0.40	160.1	1.50	0.65×
500	94.0	2.60	96.0	2.08	1.02×
1000	46.1	9.13	51.7	3.90	1.12×
2000	18.6	34.68	22.5	12.27	1.21×

En todas las pruebas el estado final de la versión paralela fue idéntico bit a bit al de la secuencial (columna `resultado_identico` del CSV).

4.1. Discusión

- **La detección de colisiones domina.** Entre el 93 % y el 99 % del tiempo del frame está en la etapa $O(N^2)$ (Tabla 3); mover y aplicar son $O(N)$ y casi despreciables. Por eso paralelizar solo el movimiento, como se pensó al inicio, no habría dado ganancia visible.

- **Costo de eliminar la condición de carrera.** La versión OpenMP con un hilo tarda cerca del doble que la secuencial ($S_{\text{sec}} \approx 0.5$), porque revisa cada par dos veces. Por eso el speedup contra 1 hilo (S_{1h}) es alto, pero el speedup real contra la mejor versión secuencial queda en aproximadamente la mitad.
- **Hardware.** El equipo tiene 4 núcleos físicos; pasar de 4 a 8 hilos usa *hyper-threading*, que comparte las unidades de cómputo de cada núcleo, por lo que la ganancia de 4 a 8 hilos es menor y la eficiencia cae.
- **Temperatura.** En una laptop la frecuencia baja tras carga sostenida; en una corrida previa, $N = 4000$ (que se mide al final) salió hasta 35 % más lento que al medirlo solo con el equipo frío. Por eso se reporta el promedio de 10 repeticiones y la desviación estándar.
- **Política de espera de OpenMP.** En el screensaver el dibujo también corre en CPU (llvmpipe en WSLg). Con la espera activa por defecto, los hilos de OpenMP que “giran” tras cada región le roban núcleos al render y los FPS paralelos llegaron a ser menores que los secuenciales; con `OMP_WAIT_POLICY=passive` la versión paralela sí mejora los FPS. En el benchmark sin ventana ocurre lo contrario: `passive` empeora el tiempo porque despertar a los hilos en cada región cuesta de 0.1 a 0.4 ms.
- **FPS.** Los FPS del screensaver dependen también del dibujo, que es secuencial; por la ley de Amdahl la mejora en FPS es menor que la mejora en el tiempo de simulación. Con N pequeño ($N = 250$) la versión paralela incluso da menos FPS (Tabla 4): la simulación tarda menos de 0.5 ms y el costo de crear y sincronizar el equipo de hilos en cada región supera la ganancia. La paralelización conviene a partir de $N \approx 500$.

5. Conclusiones

1. Se implementó un screensaver en C++/SDL2 que cumple los requisitos del enunciado, con una versión secuencial y una paralela con OpenMP que se pueden comparar en vivo mediante la zona paralela.
2. La región con mayor potencial de paralelización es la detección de colisiones ($O(N^2)$), que concentra entre el 93 % y el 99 % del tiempo de simulación.
3. Reorganizar la detección para que cada hilo escriba solo sus propios datos eliminó las condiciones de carrera sin *locks* ni atómicos, y produce resultados idénticos bit a bit a la versión secuencial, a costa de duplicar el número de revisiones.
4. El speedup real contra la mejor versión secuencial está limitado por ese trabajo extra, por los 4 núcleos físicos del equipo y por el *throttling* térmico; el speedup contra la versión paralela con un hilo muestra buena escalabilidad hasta 4 hilos.
5. En una aplicación gráfica, la política de espera de los hilos de OpenMP puede importar tanto como el algoritmo: la espera activa compite con el renderizado por software.

6. Recomendaciones

- Implementar una versión paralela que conserve el triángulo $i < j$ usando un buffer privado por hilo y una reducción final, con `schedule(dynamic)` o `guided` por el desbalance del triángulo, y compararla contra la versión actual.

- Reducir la complejidad de la detección con una rejilla espacial (*uniform grid*), de modo que cada chucho solo revise a sus vecinos.
- Medir en un equipo con GPU para que el dibujo no compita por la CPU con los hilos de OpenMP.

Referencias

- [1] G. M. Amdahl, “Validity of the single processor approach to achieving large scale computing capabilities,” en *Proc. AFIPS Spring Joint Computer Conference*, 1967, pp. 483–485.
- [2] B. Chapman, G. Jost y R. van der Pas, *Using OpenMP: Portable Shared Memory Parallel Programming*. Cambridge, MA: MIT Press, 2008.
- [3] I. Foster, *Designing and Building Parallel Programs*. Reading, MA: Addison-Wesley, 1995.
- [4] OpenMP Architecture Review Board, *OpenMP Application Programming Interface, Version 5.2*, 2021. [En línea]. Disponible: <https://www.openmp.org/specifications/>
- [5] P. Pacheco, *An Introduction to Parallel Programming*. Burlington, MA: Morgan Kaufmann, 2011.
- [6] SDL Project, *Simple DirectMedia Layer 2 – Documentation Wiki*. [En línea]. Disponible: <https://wiki.libsdl.org/SDL2/>

A. Anexo 1 – Diagrama de flujo del programa

El programa no pide datos de forma interactiva durante la ejecución: los parámetros se leen de la línea de comandos y, durante la animación, el usuario puede ajustar la zona paralela con las flechas o salir con Esc o cerrando la ventana. Los bloques azules son secciones paralelas; cada una termina en una barrera implícita.

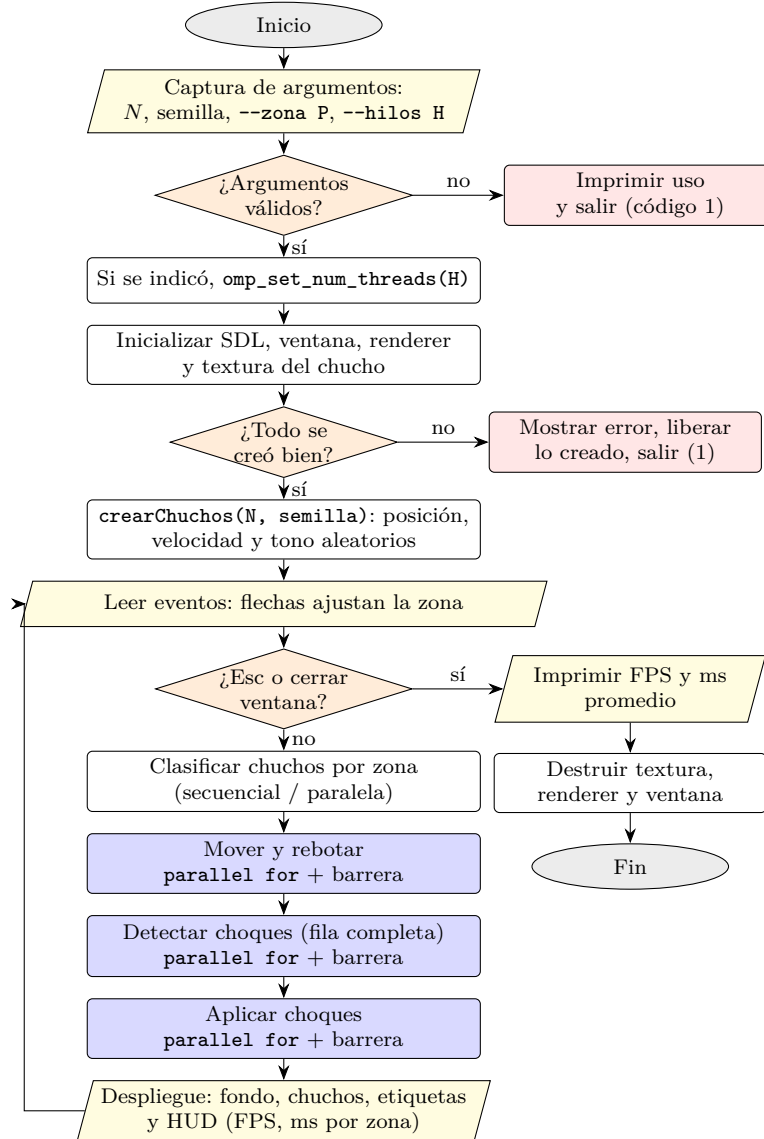


Figura 3: Diagrama de flujo del screensaver. En la zona secuencial las mismas etapas azules corren con un solo hilo (`if(false)`).

El benchmark sigue el mismo esquema sin ventana: valida sus argumentos (`--n`, `--hilos`, `--frames`, `--reps`, `--semilla`, `--salida`), y para cada N y cantidad de hilos repite la simulación, mide cada etapa con `omp_get_wtime()`, verifica el estado final contra el secuencial y escribe la tabla en consola y los CSV.

B. Anexo 2 – Catálogo de funciones

Función / clase	Entradas	Salidas	Descripción
src/main.cpp main	int argc, char* argv[]: argumentos del programa	int: código de salida	Valida argumentos, inicializa SDL, crea la simulación y corre el ciclo eventos → simulación → dibujo; al salir imprime promedios y libera recursos.
parsearArgumentos	argc, argv; Opciones& op	bool: argumentos válidos; llena op (n, semilla, zona, hilos)	Lee N y semilla posicionales y las opciones <code>--zona</code> (0–100) y <code>--hilos</code> (1–1024).
procesarEventos	ZonaParalela& zona	bool: seguir ejecutando	Atiende cerrar ventana y Esc; las flechas cambian el tamaño de la zona.
src/argumentos.cpp parsearEntero	const char* texto; long min, max	bool; long& salida	Convierte texto a entero con <code>strtol</code> , rechazando texto extra, desbordes y valores fuera de rango.
parsearListaEnteros	const char* texto (ej. "1,2,4"); min, max	bool; vector<int>& salida	Separa por comas y valida cada elemento con <code>parsearEntero</code> .
src/chucho.cpp, src/color.cpp struct Chucho	–	–	Estado de un chucho: x , y , vx , vy (float), <code>tono</code> , <code>tinte</code> (RGB) y <code>pausaColor</code> .
crearChuchos	int n; unsigned semilla	vector<Chucho>	Crea n chuchos con posición, velocidad (60–240 px/s, ángulo aleatorio) y tono pseudoaleatorios (<code>mt19937</code>).
colorDesdeTono	float tono (grados)	Color (RGB)	Normaliza el tono a $[0, 360]$ y lo convierte con <code>hsvARgb</code> usando la saturación y brillo de <code>config.h</code> .
hsvARgb	float h, s, v	Color	Conversión estándar de HSV a RGB.
src/zona.cpp ZonaParalela	int porcentaje	objeto	Franja derecha de la pantalla procesada con OpenMP.
ajustar	int delta	–	Suma <code>delta</code> al porcentaje, acotado a $[0, 100]$.
inicioX	–	int: píxel donde empieza la zona	$640 - 640 \cdot p/100$.
contiene	const Chucho& c	bool	Verdadero si el centro del chucho está en la zona.
src/simulacion.cpp moverChuchos	vector<Chucho>&; const vector<int>& indices; float dt; bool paralelo	chuchos modificados	Integra la posición, rebota en bordes y descuenta la pausa de color. Sección paralela (<code>parallel for static if(paralelo)</code>).
rebotarEje	float& pos, float& vel; min, max	pos, vel corregidos	Refleja la posición en el borde y fuerza la velocidad hacia adentro.
medirMs	función f	double: ms	Ejecuta <code>f</code> y mide su duración con <code>omp_get_wtime</code> .

Función / clase	Entradas	Salidas	Descripción
<code>Simulacion::paso</code>	<code>float dt; const ZonaParalela&</code>	<code>TiemposFrame</code> (ms secuencial y paralelo)	Clasifica por zona y ejecuta mover, detectar y aplicar para cada zona, midiendo cada una.
<code>Simulacion::clasificar</code>	<code>const ZonaParalela&</code>	listas de índices por zona	Separa los chuchos según la zona de su centro.
src/colisiones.cpp (ManejadorColisiones)			
<code>preparar</code>	<code>int n</code>	buffers de tamaño n	Ajusta el tamaño de los buffers (se reutilizan entre frames).
<code>detectar</code>	<code>const vector<Chucho>& indices; bool paralelo</code>	buffers <code>sumaVx_</code> , <code>sumaVy_</code> , <code>sumaRapidez_</code> , <code>choques_</code>	Cada chucho revisa a todos los demás y escribe solo su entrada del buffer, usando acumuladores locales. Sección paralela , sin condiciones de carrera.
<code>detectarTriangulo</code>	<code>const vector<Chucho>&</code>	misimos buffers	Versión secuencial de referencia: recorre solo $i < j$ y escribe en i y j . Resultado idéntico a <code>detectar</code> .
<code>aplicar</code>	<code>vector<Chucho>& indices; bool paralelo</code>	chuchos con nueva velocidad y tono	Aplica dirección y rapidez promedio de los compañeros; gira el tono si pasó la pausa. Sección paralela .
<code>seTraslapan</code>	<code>const Chucho& a, b</code>	<code>bool</code>	Prueba de cajas envolventes de 56×52 .
<code>seAcercan</code>	<code>const Chucho& a, b</code>	<code>bool</code>	Producto punto de posición y velocidad relativas negativo.
src/estadisticas.cpp, src/hud.cpp, src/render.cpp, src/texto.cpp, src/sprite.cpp			
<code>Estadisticas::registrarFrame</code>	<code>float dt; const TiemposFrame&</code>	valores actuales y totales	Acumula FPS y ms por zona; recalcula los valores del HUD cada 0.5 s.
<code>fpsPromedio, tiemposPromedio</code>	—	<code>float, TiemposFrame</code>	Promedios de toda la corrida, impresos al salir.
<code>crearTexturaChucho</code>	<code>SDL_Renderer*</code>	<code>SDL_Texture*</code> (o <code>nullptr</code>)	Genera la textura de 64×64 a partir de un bitmap de 16×16 escalado 4 veces.
<code>dibujarFondo</code>	<code>SDL_Renderer*; const ZonaParalela&</code>	—	Pinta la zona secuencial, la zona paralela (azul) y la línea divisoria.
<code>dibujarChuchos</code>	<code>SDL_Renderer*; SDL_Texture*; const vector<Chucho>&</code>	—	Dibuja cada chucho con su tinte, volteado según su dirección.
<code>dibujarEtiquetasZonas</code>	<code>renderer; zona; int chuchos por zona</code>	—	Escribe “SECUENCIAL (n)” y “OPENMP (n)” al pie de cada zona.
<code>dibujarHud</code>	<code>SDL_Renderer*; const DatosHud&</code>	—	Recuadro con FPS, N , hilos, zona y ms por zona.
<code>dibujarTexto, anchoTexto, altoTexto</code>	<code>texto; posición; escala</code>	— / <code>int</code>	Fuente de píxeles propia (A–Z, 0–9 y símbolos).
bench/ (programa de benchmark)			
<code>medir</code>	<code>const ConfigMedicion& (n, versión, hilos, frames, calentamiento, repeticiones, semilla)</code>	<code>Resultado Medicion</code>	Corre la simulación sin ventana y mide ms por frame de cada etapa en cada repetición.
<code>pasoMedido</code>	<code>chuchos; índices; colisiones; versión</code>	<code>TiemposEtapas& acumulado</code>	Un frame con dt fijo, midiendo mover, detectar y aplicar.
<code>misimosEstados</code>	<code>dos vector<Chucho></code>	<code>bool</code>	Compara campo por campo, bit a bit.
<code>armarReporte</code>	<code>vector<Resultado Medicion></code>	<code>vector<Fila Reporte></code>	Calcula speedup (vs. secuencial y vs. 1 hilo), eficiencia y verificación.

Función / clase	Entradas	Salidas	Descripción
<code>imprimirTabla,</code> <code>escribirCsv,</code> <code>escribirCsvDetalle</code>	filas; ruta	consola / archivos CSV	Despliegue de resultados.

C. Anexo 3 – Bitácora de pruebas

Cada prueba es una combinación de N y versión, con 10 mediciones (M1–M10). Cada medición es el promedio de ms por frame de 30 frames. S es el speedup contra la versión secuencial y E la eficiencia (S/p).

Tabla 6: Bitácora de mediciones, $N = 500$ (ms por frame de simulación)

Medición	Secuencial	OMP 1h	OMP 2h	OMP 4h	OMP 8h
M1	0.531	0.958	0.593	0.337	0.179
M2	0.521	0.977	0.509	0.354	0.224
M3	0.511	0.969	0.504	0.344	0.248
M4	0.510	1.002	0.516	0.252	0.185
M5	0.514	1.018	0.513	0.316	0.272
M6	0.473	1.014	0.575	0.348	0.189
M7	0.489	1.116	0.683	0.347	0.293
M8	0.501	1.108	0.516	0.457	0.196
M9	0.474	1.049	0.509	0.299	0.449
M10	0.504	1.102	0.499	0.301	0.294
Promedio	0.503	1.031	0.542	0.335	0.253
Desv. estándar	0.019	0.059	0.059	0.053	0.082
Speedup S	1.000	0.488	0.928	1.500	1.988
Eficiencia E	1.000	0.488	0.464	0.375	0.248

Tabla 7: Bitácora de mediciones, $N = 1000$ (ms por frame de simulación)

Medición	Secuencial	OMP 1h	OMP 2h	OMP 4h	OMP 8h
M1	2.012	3.851	1.906	1.413	0.985
M2	1.980	3.785	1.984	1.498	1.719
M3	1.950	3.912	2.149	1.510	1.114
M4	1.856	3.965	1.978	1.498	1.082
M5	1.868	3.687	1.972	1.530	0.971
M6	1.899	3.730	1.974	1.734	0.996
M7	1.919	3.791	1.976	1.503	0.941
M8	1.873	3.693	1.993	1.329	0.994
M9	1.968	3.652	2.181	1.476	0.909
M10	1.935	3.736	2.527	1.518	0.928
Promedio	1.926	3.780	2.064	1.501	1.064
Desv. estándar	0.052	0.102	0.184	0.102	0.239
Speedup S	1.000	0.509	0.933	1.284	1.810
Eficiencia E	1.000	0.509	0.467	0.321	0.226

Tabla 8: Bitácora de mediciones, $N = 2000$ (ms por frame de simulación)

Medición	Secuencial	OMP 1h	OMP 2h	OMP 4h	OMP 8h
M1	8.192	16.629	9.388	6.111	7.384
M2	7.982	16.114	8.793	6.139	4.091
M3	8.152	18.948	9.281	6.364	4.059
M4	8.205	16.619	8.506	6.081	4.034
M5	7.915	15.961	8.902	6.166	4.875
M6	9.379	16.012	9.115	6.222	4.042
M7	7.861	16.495	8.605	6.439	4.126
M8	8.065	15.946	9.275	5.953	5.681
M9	8.178	16.018	9.365	6.186	4.717
M10	8.227	15.979	9.024	6.006	4.235
Promedio	8.216	16.472	9.025	6.167	4.724
Desv. estándar	0.429	0.913	0.316	0.149	1.075
Speedup S	1.000	0.499	0.910	1.332	1.739
Eficiencia E	1.000	0.499	0.455	0.333	0.217

Tabla 9: Bitácora de mediciones, $N = 4000$ (ms por frame de simulación)

Medición	Secuencial	OMP 1h	OMP 2h	OMP 4h	OMP 8h
M1	34.522	73.285	38.507	32.071	25.752
M2	34.324	73.992	41.193	33.453	26.268
M3	34.959	72.982	39.310	33.519	27.013
M4	34.832	98.165	40.834	31.850	35.185
M5	40.255	67.041	44.904	35.607	22.660
M6	37.835	68.132	46.083	32.748	25.451
M7	38.166	75.656	60.025	30.676	26.774
M8	35.551	71.247	62.433	30.534	29.899
M9	34.662	68.979	46.428	37.620	26.723
M10	36.764	66.862	46.136	36.247	28.608
Promedio	36.187	73.634	46.585	33.432	27.433
Desv. estándar	1.995	9.150	8.269	2.381	3.326
Speedup S	1.000	0.491	0.777	1.082	1.319
Eficiencia E	1.000	0.491	0.388	0.271	0.165

Captura de las mediciones

Salida en consola de `make bench`:

Núcleos disponibles: 8 OMP_WAIT_POLICY: (sin definir), OMP_PROC_BIND: (sin definir) Frames: 30 (+5 de calentamiento), repeticiones: 10, semilla: 7													
N	version	hilos	ms/frame	desv	min	mover	detectar	aplicar	S(sec)	S(1h)	efic	identico	
500	secuencial	-	0.503	0.019	0.473	0.003	0.468	0.032	1.00x	-	1.00	si	
500	openmp	1	1.031	0.060	0.958	0.003	0.993	0.035	0.49x	1.00x	0.49	si	
500	openmp	2	0.542	0.059	0.500	0.002	0.522	0.018	0.93x	1.90x	0.46	si	
500	openmp	4	0.335	0.053	0.252	0.002	0.322	0.011	1.50x	3.08x	0.37	si	
500	openmp	8	0.253	0.082	0.179	0.008	0.204	0.040	1.99x	4.08x	0.25	si	
1000	secuencial	-	1.926	0.052	1.856	0.006	1.856	0.064	1.00x	-	1.00	si	
1000	openmp	1	3.780	0.102	3.653	0.007	3.710	0.064	0.51x	1.00x	0.51	si	
1000	openmp	2	2.064	0.184	1.906	0.005	2.022	0.037	0.93x	1.83x	0.47	si	
1000	openmp	4	1.501	0.102	1.329	0.006	1.465	0.031	1.28x	2.52x	0.32	si	
1000	openmp	8	1.064	0.239	0.909	0.046	0.910	0.108	1.81x	3.55x	0.23	si	

2000	secuencial	-		8.216	0.429	7.861		0.012	8.075	0.129		1.00x	-	1.00	si
2000	openmp	1		16.472	0.913	15.946		0.012	16.330	0.130		0.50x	1.00x	0.50	si
2000	openmp	2		9.026	0.316	8.506		0.007	8.945	0.073		0.91x	1.83x	0.46	si
2000	openmp	4		6.167	0.149	5.953		0.007	6.102	0.057		1.33x	2.67x	0.33	si
2000	openmp	8		4.724	1.075	4.034		0.164	4.246	0.314		1.74x	3.49x	0.22	si
4000	secuencial	-		36.187	1.995	34.324		0.029	35.867	0.291		1.00x	-	1.00	si
4000	openmp	1		73.634	9.150	66.862		0.029	73.319	0.287		0.49x	1.00x	0.49	si
4000	openmp	2		46.585	8.269	38.507		0.021	46.385	0.179		0.78x	1.58x	0.39	si
4000	openmp	4		33.433	2.381	30.534		0.032	33.106	0.294		1.08x	2.20x	0.27	si
4000	openmp	8		27.433	3.326	22.660		0.766	25.537	1.130		1.32x	2.68x	0.16	si

S(sec) = speedup contra la version secuencial (triangulo i<j)
 S(1h) = speedup contra OpenMP con 1 hilo
 efic = S(sec) / hilos

Resultados guardados en resultados/benchmark.csv
 Mediciones individuales guardadas en resultados/benchmark_detalle.csv



Figura 4: Screensaver tras 10 segundos de simulación con choques.

D. Material suplementario

- Repositorio: <https://github.com/anthonylouschwank/Proyecto-Paralela>
- Compilación y ejecución (Ubuntu/WSL, requiere libSDL2-dev):

```

make                                # screensaver y benchmark
make run N=2000 ARGS="7 --zona 100" # screensaver
make bench                          # benchmark -> resultados/*.csv
python informe/generar_tablas.py    # tablas de este informe

```